BIG DATA TECHNOLOGY LAB (BDTL) · PRACTICAL REPORT

Bucket 9 · Apache Spark Experiments 3 and 4

Submitted by

Adesh Patankar

Experiment	Title and learning objective
Experiment 3	**Connect Spark with Hadoop (HDFS)** Configure Spark to read/write data from HDFS. Verify with a small read/write operation.
Experiment 4	**Run Sample Application** Execute sample Spark examples (e.g., Pi estimation or WordCount example) to confirm correct setup.

Platform: Apache Spark 3.5.8 (Scala 2.12, Java 8, local mode) on Apache Hadoop 3.3.6 (pseudo-distributed HDFS + YARN), Ubuntu 24.04 LTS.
All execution evidence in this report consists of screenshots of the actual terminal session in which the experiments were run.

Contents

	Section
Experiment 3	**Connect Spark with Hadoop (HDFS)**
	A. Experiment Details B. Aim C. Theory D. Requirements E. Procedure and Implementation F. Execution Screenshots G. Output and Results H. Result I. Observations and Troubleshooting
Experiment 4	**Run Sample Application**
	A. Experiment Details B. Aim C. Theory D. Requirements E. Procedure and Implementation F. Execution Screenshots G. Output and Results H. Result I. Observations and Troubleshooting

Bucket 9 – Experiment 3: Connect Spark with Hadoop (HDFS)

A. Experiment Details

Field	Details
Bucket	9
Sub-experiment No.	3
Title	Connect Spark with Hadoop (HDFS)
Description / Learning Objective (bucket list)	Configure Spark to read/write data from HDFS. Verify with a small read/write operation.
Student	Adesh Patankar
Subject	Big Data Technology Lab (BDTL)
Tool / Technology	Apache Spark 3.5.8 (Scala shell, PySpark, spark-submit) with Apache Hadoop 3.3.6 HDFS

B. Aim

To configure Spark to use the Hadoop cluster configuration so that it reads and writes HDFS, and to verify the connection by reading a text file from HDFS, writing an RDD and a DataFrame back to HDFS, and checking the results with HDFS commands.

C. Theory

C.1 Apache Spark

Apache Spark is an open-source distributed data-processing engine. A **driver** program creates a `SparkContext`/`SparkSession`, which connects to a cluster manager (here `local[*]`, i.e. all cores of one machine; YARN, standalone or Kubernetes on a cluster) and runs **executors** that process partitions of the data in parallel. Data is represented as **RDDs** (resilient distributed datasets) or **DataFrames**; *transformations* are lazy and build a DAG, *actions* execute it. Spark keeps intermediate data in memory, which makes it much faster than disk-based MapReduce for iterative and interactive work. It offers APIs in Scala, Java, Python (PySpark) and R.

C.2 How Spark finds HDFS

Spark does not have its own storage; it uses the Hadoop `FileSystem` API, which is included in the *pre-built for Hadoop 3* package. When `HADOOP_CONF_DIR` is set in `$SPARK_HOME/conf/spark-env.sh`, Spark loads `core-site.xml` and `hdfs-site.xml` from that directory. The property `fs.defaultFS` (`hdfs://localhost:9000`) then becomes Spark's default file system, so a path without a scheme such as `/bdt/bucket9/sample.txt` refers to HDFS. A full URI (`hdfs://localhost:9000/...`) works in either case. `YARN_CONF_DIR` additionally allows `--master yarn`.

Operation	API
Read text	`sc.textFile(path)` – one RDD element per line
Write text	`rdd.saveAsTextFile(dir)` – one `part-NNNNN` file per partition + `_SUCCESS`; fails if the directory exists
Write CSV	`df.write.mode('overwrite').option('header', True).csv(dir)`
Read CSV	`spark.read.option('header', True).option('inferSchema', True).csv(dir)`

D. Requirements

Item	Used in this experiment
Host machine	Apple MacBook Air (Apple Silicon, arm64) running macOS
Virtual machine	Multipass VM `bigdata` – Ubuntu 24.04 LTS (arm64), 2 vCPU, 4 GB RAM, 2 GB swap
Java	OpenJDK 8 (1.8.0_504), `JAVA_HOME=/usr/lib/jvm/java-8-openjdk-arm64`
Spark	Apache Spark 3.5.8, pre-built for Hadoop 3 (`spark-3.5.8-bin-hadoop3`), Scala 2.12.18, installed in `/home/ubuntu/spark-3.5.8-bin-hadoop3`; run in local mode (`master = local[*]`)
Hadoop	Apache Hadoop 3.3.6, pseudo-distributed: HDFS at `hdfs://localhost:9000` and YARN
Hadoop daemons	NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager, JobHistoryServer (started by the runner if needed)
Input	`bucket9/data/sample.txt` (3 lines) uploaded to `/bdt/bucket9/`
Runner	`bdt/bucket9/run_bucket9.sh` – 7 parts covering Sub-Experiments 1–4; every command is printed (green) before it runs

E. Procedure and Implementation

Step 1 – Configure Spark for Hadoop

`$SPARK_HOME/conf/spark-env.sh`

```
export HADOOP_CONF_DIR=/home/ubuntu/hadoop-3.3.6/etc/hadoop
export YARN_CONF_DIR=/home/ubuntu/hadoop-3.3.6/etc/hadoop
```

Step 2 – Upload the sample file to HDFS

`bucket9/data/sample.txt`

```
Apache Spark connects to Hadoop HDFS
Spark reads data from HDFS and writes results back to HDFS
HDFS stores data and Spark processes data in memory
```

`Shell`

```
grep -A1 fs.defaultFS $HADOOP_HOME/etc/hadoop/core-site.xml
jps
hdfs dfs -mkdir -p /bdt/bucket9
hdfs dfs -put -f data/sample.txt /bdt/bucket9/
```

```
hdfs dfs -ls /bdt/bucket9
hdfs dfs -cat /bdt/bucket9/sample.txt
```

Step 3 – Read/write program

bucket9/py/hdfs_read_write.py

```
# Bucket 9 | Sub-Exp 3 : read and write HDFS data from Spark
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Bucket9-HDFS-ReadWrite").getOrCreate()
sc = spark.sparkContext
print("fs.defaultFS used by Spark :", sc._jsc.hadoopConfiguration().get("fs.defaultFS"))

# 1. READ a text file from HDFS (no scheme needed: the default file system is HDFS)
lines = sc.textFile("/bdt/bucket9/sample.txt")
print("Lines read from HDFS       :", lines.count())

# 2. WRITE an RDD back to HDFS as a text file
lines.map(lambda l: l.upper()).saveAsTextFile("/bdt/bucket9/output_text")
print("RDD written to             : /bdt/bucket9/output_text")

# 3. WRITE a DataFrame to HDFS as CSV, then READ it back
products = spark.createDataFrame(
    [(1, "Laptop", 55000), (2, "Mouse", 700), (3, "Keyboard", 1500), (4, "Monitor", 12000)],
    ["id", "product", "price"])
products.write.mode("overwrite").option("header",
True).csv("hdfs://localhost:9000/bdt/bucket9/output_csv")
print("DataFrame written to       : hdfs://localhost:9000/bdt/bucket9/output_csv")

back = spark.read.option("header", True).option("inferSchema", True).csv("/bdt/bucket9/output_csv")
print("Rows read back from HDFS   :", back.count())
back.orderBy("id").show()
spark.stop()
```

Run and verify

```
spark-submit py/hdfs_read_write.py
hdfs dfs -ls /bdt/bucket9 /bdt/bucket9/output_text /bdt/bucket9/output_csv
hdfs dfs -cat /bdt/bucket9/output_text/part-*
hdfs dfs -cat /bdt/bucket9/output_csv/part-*
```

F. Execution Screenshots

Screenshots of the Ubuntu VM terminal (`ubuntu@bigdata`) while `run_bucket9.sh` was executing. Only empty margins were trimmed.

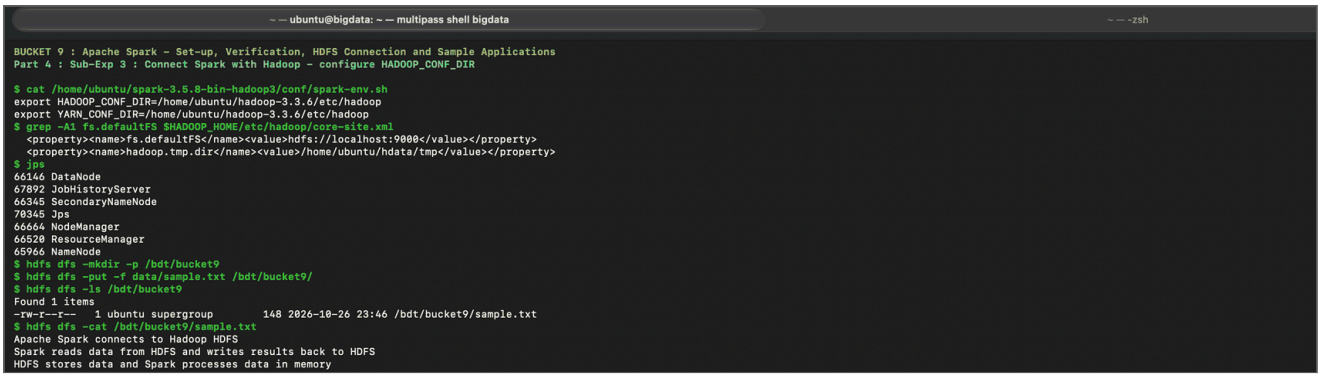


Figure 3.1: Configuring HADOOP_CONF_DIR and uploading the input to HDFS

`spark-env.sh` points `HADOOP_CONF_DIR` and `YARN_CONF_DIR` to `hadoop-3.3.6/etc/hadoop`, whose `core-site.xml` sets `fs.defaultFS = hdfs://localhost:9000`. `jps` shows all Hadoop daemons running. `sample.txt` (148 B) is uploaded to `/bdt/bucket9` and read back.

~ — ubuntu@bigdata: ~ — multipass shell bigdata

```
BUCKET 9 : Apache Spark – Set-up, Verification, HDFS Connection and Sample Applications
Part 5 : Sub-Exp 3 : Small read/write operation on HDFS from Spark

$ cat -n py/hdfs_read_write.py
     1  # Bucket 9 | Sub-Exp 3 : read and write HDFS data from Spark
     2  from pyspark.sql import SparkSession
     3
     4  spark = SparkSession.builder.appName("Bucket9-HDFS-ReadWrite").getOrCreate()
     5  sc = spark.sparkContext
     6  print("fs.defaultFS used by Spark :", sc._jsc.hadoopConfiguration().get("fs.defaultFS"))
     7
     8  # 1. READ a text file from HDFS (no scheme needed: the default file system is HDFS)
     9  lines = sc.textFile("/bdt/bucket9/sample.txt")
    10  print("Lines read from HDFS       :", lines.count())
    11
    12  # 2. WRITE an RDD back to HDFS as a text file
    13  lines.map(lambda l: l.upper()).saveAsTextFile("/bdt/bucket9/output_text")
    14  print("RDD written to             : /bdt/bucket9/output_text")
    15
    16  # 3. WRITE a DataFrame to HDFS as CSV, then READ it back
    17  products = spark.createDataFrame(
    18      [(1, "Laptop", 55000), (2, "Mouse", 700), (3, "Keyboard", 1500), (4, "Monitor", 12000)],
    19      ["id", "product", "price"])
    20  products.write.mode("overwrite").option("header", True).csv("hdfs://localhost:9000/bdt/bucket9/output_csv")
    21  print("DataFrame written to       : hdfs://localhost:9000/bdt/bucket9/output_csv")
    22
    23  back = spark.read.option("header", True).option("inferSchema", True).csv("/bdt/bucket9/output_csv")
    24  print("Rows read back from HDFS   :", back.count())
    25  back.orderBy("id").show()
    26  spark.stop()

$ spark-submit py/hdfs_read_write.py
26/10/26 23:47:11 WARN Utils: Your hostname, bigdata resolves to a loopback address: 127.0.1.1; using 192.168.252.2 instead (on interface enp0s1)
26/10/26 23:47:11 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
26/10/26 23:47:11 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
fs.defaultFS used by Spark : hdfs://localhost:9000
Lines read from HDFS       : 3
RDD written to             : /bdt/bucket9/output_text
DataFrame written to       : hdfs://localhost:9000/bdt/bucket9/output_csv
Rows read back from HDFS   : 4
+---+--------+-----+
| id| product|price|
+---+--------+-----+
|  1|  Laptop|55000|
|  2|   Mouse|  700|
|  3|Keyboard| 1500|
|  4| Monitor|12000|
+---+--------+-----+


--- Verification with HDFS commands ---
$ hdfs dfs -ls /bdt/bucket9 /bdt/bucket9/output_text /bdt/bucket9/output_csv
Found 3 items
drwxr-xr-x   - ubuntu supergroup          0 2026-10-26 23:47 /bdt/bucket9/output_csv
drwxr-xr-x   - ubuntu supergroup          0 2026-10-26 23:47 /bdt/bucket9/output_text
-rw-r--r--   1 ubuntu supergroup        148 2026-10-26 23:46 /bdt/bucket9/sample.txt
Found 3 items
-rw-r--r--   1 ubuntu supergroup          0 2026-10-26 23:47 /bdt/bucket9/output_text/_SUCCESS
-rw-r--r--   1 ubuntu supergroup         96 2026-10-26 23:47 /bdt/bucket9/output_text/part-00000
-rw-r--r--   1 ubuntu supergroup         52 2026-10-26 23:47 /bdt/bucket9/output_text/part-00001
Found 3 items
-rw-r--r--   1 ubuntu supergroup          0 2026-10-26 23:47 /bdt/bucket9/output_csv/_SUCCESS
-rw-r--r--   1 ubuntu supergroup         44 2026-10-26 23:47 /bdt/bucket9/output_csv/part-00000-cbe4b423-12f5-475a-a53f-eb1efa52de60-c000.csv
-rw-r--r--   1 ubuntu supergroup         49 2026-10-26 23:47 /bdt/bucket9/output_csv/part-00001-cbe4b423-12f5-475a-a53f-eb1efa52de60-c000.csv
$ hdfs dfs -cat /bdt/bucket9/output_text/part-*
APACHE SPARK CONNECTS TO HADOOP HDFS
SPARK READS DATA FROM HDFS AND WRITES RESULTS BACK TO HDFS
HDFS STORES DATA AND SPARK PROCESSES DATA IN MEMORY
$ hdfs dfs -cat /bdt/bucket9/output_csv/part-*
id,product,price
1,Laptop,55000
2,Mouse,700
```

Figure 3.2: Spark reading from and writing to HDFS, verified with HDFS commands

The program prints `fs.defaultFS used by Spark : hdfs://localhost:9000`, reads **3 lines** from HDFS, writes the upper-cased lines to `output_text`, writes a 4-row DataFrame as CSV to `output_csv` and reads it back (**4 rows**, shown as a table). The HDFS listing shows both output directories, each with `_SUCCESS` and two part files (one per partition); `-cat` prints the upper-case text and the CSV with its header (the screenshot ends after the first two CSV rows; the other two are in the second part file and appear in the table read back above).

G. Output and Results

Operation	Location in HDFS	Result
Default file system	–	`hdfs://localhost:9000` (from HADOOP_CONF_DIR)
Read text file	`/bdt/bucket9/sample.txt`	3 lines
Write RDD (text)	`/bdt/bucket9/output_text`	_SUCCESS, part-00000 (96 B), part-00001 (52 B); 3 upper-case lines
Write DataFrame (CSV)	`/bdt/bucket9/output_csv`	_SUCCESS, part-00000 (44 B), part-00001 (49 B), header `id,product,price`
Read CSV back	`/bdt/bucket9/output_csv`	4 rows: Laptop 55000, Mouse 700, Keyboard 1500, Monitor 12000

H. Result

> Spark was connected to Hadoop by setting `HADOOP_CONF_DIR`; it used `hdfs://localhost:9000` as its default file system, read a file from HDFS, and wrote both a text output and a CSV output to HDFS that were confirmed with `hdfs dfs -ls` and `-cat`.

I. Observations and Troubleshooting

Observations

- The warnings *Your hostname, bigdata resolves to a loopback address ... using 192.168.252.2 instead* and *Unable to load native-hadoop library ... using builtin-java classes* are harmless: Spark binds to the VM's network address and uses Java implementations instead of native Hadoop libraries.
- Paths without a scheme (`/bdt/bucket9/...`) and full URIs (`hdfs://localhost:9000/...`) both reached HDFS.
- Each output is a *directory*, not a single file: one part file per partition, plus an empty `_SUCCESS` marker.
- CSV part files have generated names (`part-00000-<uuid>-c000.csv`); the header is written in every part file and skipped again when reading with `header=True`.
- The runner deletes old output directories first, because `saveAsTextFile` fails if the directory already exists.

Troubleshooting

- The part ran without errors; no troubleshooting was required.

Bucket 9 – Experiment 4: Run Sample Application

A. Experiment Details

Field	Details
Bucket	9
Sub-experiment No.	4
Title	Run Sample Application
Description / Learning Objective (bucket list)	Execute sample Spark examples (e.g., Pi estimation or WordCount example) to confirm correct setup.
Student	Adesh Patankar
Subject	Big Data Technology Lab (BDTL)
Tool / Technology	Apache Spark 3.5.8 (Scala shell, PySpark, spark-submit) with Apache Hadoop 3.3.6 HDFS

B. Aim

To confirm that the Spark set-up works end to end by running the example applications shipped with Spark: **SparkPi** (Scala, Pi estimation) and the Python **WordCount** example on a file stored in HDFS.

C. Theory

C.1 Apache Spark

Apache Spark is an open-source distributed data-processing engine. A **driver** program creates a `SparkContext`/`SparkSession`, which connects to a cluster manager (here `local[*]`, i.e. all cores of one machine; YARN, standalone or Kubernetes on a cluster) and runs **executors** that process partitions of the data in parallel. Data is represented as **RDDs** (resilient distributed datasets) or **DataFrames**; *transformations* are lazy and build a DAG, *actions* execute it. Spark keeps intermediate data in memory, which makes it much faster than disk-based MapReduce for iterative and interactive work. It offers APIs in Scala, Java, Python (PySpark) and R.

C.2 Bundled examples

The Spark distribution contains example programs in `$SPARK_HOME/examples`: compiled Scala/Java examples in `examples/jars/spark-examples_2.12-3.5.8.jar` and source code in `examples/src/main/{scala,java,python,r}`. `run-example <Class>` is a wrapper around `spark-submit` for the jar examples; Python examples are run with `spark-submit file.py`.

C.3 SparkPi – Monte Carlo estimation

Random points (x, y) are generated uniformly in the square $[-1, 1] \times [-1, 1]$. The fraction falling inside the unit circle ($x^2 + y^2 \leq 1$) approximates the area ratio $\pi/4$, so $\pi \approx 4 \times$ inside / total. The argument (`10`) is the number of partitions (slices); each slice processes 100,000 points in a separate task, and the counts are combined with `reduce`. More points give a closer estimate.

C.4 WordCount example

`wordcount.py` reads a file into lines, splits them on spaces (`flatMap`), maps each word to (word, 1) and sums with `reduceByKey`, then `collect()`s and prints `word: count`.

D. Requirements

Item	Used in this experiment
Host machine	Apple MacBook Air (Apple Silicon, arm64) running macOS
Virtual machine	Multipass VM `bigdata` – Ubuntu 24.04 LTS (arm64), 2 vCPU, 4 GB RAM, 2 GB swap
Java	OpenJDK 8 (1.8.0_504), `JAVA_HOME=/usr/lib/jvm/java-8-openjdk-arm64`
Spark	Apache Spark 3.5.8, pre-built for Hadoop 3 (`spark-3.5.8-bin-hadoop3`), Scala 2.12.18, installed in `/home/ubuntu/spark-3.5.8-bin-hadoop3`; run in local mode (`master = local[*]`)
Hadoop	Apache Hadoop 3.3.6, pseudo-distributed: HDFS at `hdfs://localhost:9000` and YARN
Examples	`$SPARK_HOME/examples/jars/spark-examples_2.12-3.5.8.jar` and `examples/src/main/python/wordcount.py`
Input	`/bdt/bucket9/sample.txt` in HDFS (uploaded in Experiment 3)
Runner	`bdt/bucket9/run_bucket9.sh` – 7 parts covering Sub-Experiments 1–4; every command is printed (green) before it runs

E. Procedure and Implementation

bucket9/data/sample.txt

```
Apache Spark connects to Hadoop HDFS
Spark reads data from HDFS and writes results back to HDFS
HDFS stores data and Spark processes data in memory
```

Step 1 – Pi estimation (10 partitions)

```
run-example SparkPi 10
```

Step 2 – WordCount example on the HDFS file

```
ls $SPARK_HOME/examples/src/main/python/wordcount.py
spark-submit $SPARK_HOME/examples/src/main/python/wordcount.py /bdt/bucket9/sample.txt
```

F. Execution Screenshots

Screenshots of the Ubuntu VM terminal (`ubuntu@bigdata`) while `run_bucket9.sh` was executing. Only empty margins were trimmed.



Figure 4.1: SparkPi example (Pi estimation)

`run-example SparkPi 10` submits the example jar in local mode and prints **Pi is roughly 3.145859145859146**. The WARN about the examples jar *has been added already* appears because `run-example` passes the jar both as the application and in the class path; it is harmless.

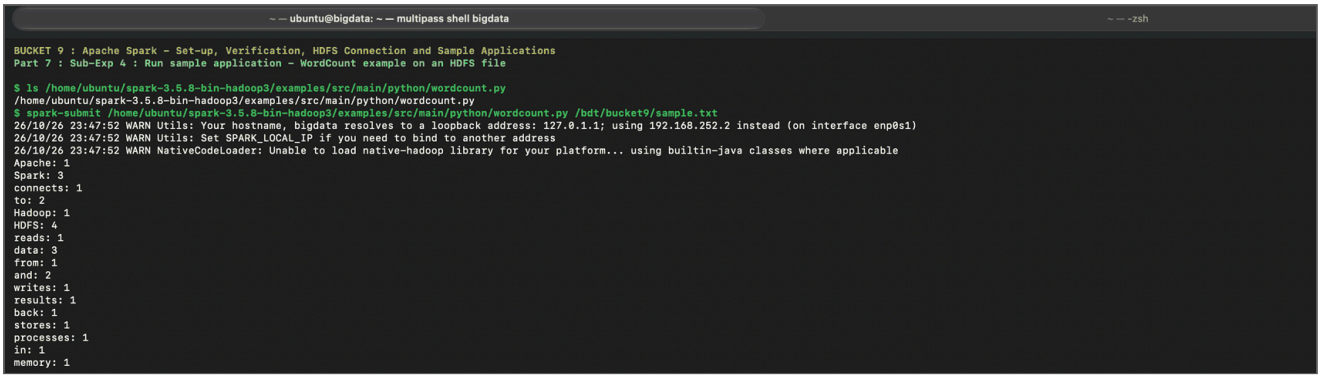


Figure 4.2: Python WordCount example on an HDFS file

The example is located in `examples/src/main/python` and run on `/bdt/bucket9/sample.txt` (read from HDFS through the default file system). Output: HDFS 4, Spark 3, data 3, to 2, and 2, and 1 for the other 12 words – 17 distinct words.

G. Output and Results

Application	Output	Check
SparkPi (10 partitions)	Pi is roughly 3.145859145859146	Error from π = 3.14159... is about 0.004 (0.14 %), as expected for 1,000,000 random points
WordCount (wordcount.py)	HDFS 4, Spark 3, data 3, to 2, and 2, others 1	17 distinct words, 26 words in total = word count of sample.txt

H. Result

> The bundled sample applications ran successfully: SparkPi estimated π as 3.1459, and the WordCount example counted the words of a file stored in HDFS correctly. This confirms that Spark is installed, configured and connected to HDFS correctly.

I. Observations and Troubleshooting

Observations

- The warnings *Your hostname, bigdata resolves to a loopback address ... using 192.168.252.2 instead* and *Unable to load native-hadoop library ... using builtin-java classes* are harmless: Spark binds to the VM's network address and uses Java implementations instead of native Hadoop libraries.
- SparkPi uses random numbers, so the digits of the estimate change from run to run.
- The example WordCount splits only on single spaces and is case-sensitive, so it is a basic demonstration; Bucket 10 adds cleaning.
- The order of the WordCount output is not sorted; it follows the partitions after `reduceByKey`.

Troubleshooting

- The part ran without errors; no troubleshooting was required.